

南京大学 操作系统（2026春）期中试卷（整理版）

学号：____
姓名：____
专业：____
总分：____

⚠ 说明

本文后半部分“解答”由 AI 生成，仅供参考；以课程标准答案/教师讲解为准。

一、问答题

阅读以下 C 程序（Linux 环境），回答三个问题。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    int pipe_fd[2];
    pipe(pipe_fd);
    int shared = 0;

    // 写者进程（fork）
    for (int i = 0; i < 2; i++) {
        if (fork() == 0) {
            close(pipe_fd[0]);          // 关闭读端
            char buf[2048];
            memset(buf, 'A' + i, sizeof(buf));
            write(pipe_fd[1], buf, sizeof(buf));
            shared = i + 1;
            close(pipe_fd[1]);          // 关闭写端
            printf("Writer%d: shared = %d\n", i, shared);
            exit(0);
        }
    }

    // 读者进程（fork）
    if (fork() == 0) {
        close(pipe_fd[1]);              // 关闭写端
        char buffer[4096];
        int n;
```

```

// 循环读取 PIPE
while ((n = read(pipe_fd[0], buffer, sizeof(buffer))) > 0) {
    printf("Reader: read %d bytes, first byte: %c, shared = %d\n",
           n, buffer[0], shared);
}
shared = 100;
_exit(0);
}

// 父进程
close(pipe_fd[0]);
close(pipe_fd[1]);
wait(NULL);
wait(NULL);
wait(NULL);
printf("Parent: shared = %d\n", shared);
return 0;
}

```

1. 读者进程（即 `fork` 创建的子进程）可能输出哪些情况（字节数、字节序列）？请说明所有可能情况并解释原因。
2. 分析该程序运行后会发生死锁吗？若会，请描述死锁情形及原因；若不会，请说明理由。
3. 程序中四个进程打印的 `shared` 变量的值分别是多少？请解释原因。

二、计算题

考虑一个采用**基于优先级的抢占式时间片轮转**调度算法的系统。调度规则如下：

- 每个进程有一个静态优先级，数值越小表示优先级越高。
- 就绪队列按优先级排序，同一优先级的进程采用时间片轮转（时间片长度固定为 2 个时间单位）。
- 进程到达时，如果其优先级高于当前正在运行的进程，则立即抢占 CPU，被抢占的进程放回其所属优先级队列末尾。
- 当进程的时间片用完但尚未完成时，它被放回当前优先级队列的末尾，并重新参与调度。
- 所有进程均为 CPU 密集型，不考虑 I/O 等待。

进程信息如下：

进程	到达时间	执行时间	优先级
P1	0	8	2
P2	1	4	1
P3	2	2	3
P4	3	6	2

要求：

1. 画出上述调度算法的时间线图（从 0 时刻开始，每个时间片内运行的进程），并写出所有进程的
执行顺序（例如：P2 → P1 → P4 → ...）。
2. 计算每个进程的响应时间（从到达时刻到首次获得 CPU 的时间间隔）以及平均响应时间。
3. 观察你得到的调度顺序和响应时间，你认为该调度算法是否公平？为什么？如果不公平，请提出
一种改进思路，使得所有进程都能更公平地获得 CPU 时间，避免某个进程长时间得不到调度。
请简要描述你的改进方法。

三、编程题

在一个机票预订系统中，共享剩余座位数 `seats`（初值为 N）和当前票价 `price`。三类并发进程如下：

- 查询进程可同时读取 `seats` 和 `price`；
- 调价进程修改 `price`，并与任何其他进程互斥；
- 预订进程读取 `price`，并原子地执行“读 `seats`；若大于零则 `seats--`”。

预订过程与调价进程互斥。预订过程与查询进程的规则是：预订进程读 `seats` 时不阻塞查询，但修改 `seats` 时必须与查询互斥（即修改期间不允许查询读取）。不允许超售。

请使用信号量（P/V 操作）实现，给出伪代码。

解答（由 AI 生成，仅供参考）

一、问答题（参考解）

| (1) 读者进程可能输出哪些情况（字节数、字节序列）？

要点：

- 两个写者分别 `write 2048` 字节到同一管道，内容全为 `'A'` 与全为 `'B'`。
- 写者先后顺序不确定：管道数据可能先 A 后 B，也可能先 B 后 A。
- `read(pipe, buffer, 4096)` 每次返回多少不固定：可能一次读到 2048，也可能一次读到 4096；也可能被拆成多次（每次返回 1~4096 的正数，累计总和为 4096）。

典型输出形态：

- 两次读：`read 2048 ... first byte A`，再 `read 2048 ... first byte B`（或相反）
- 一次读：`read 4096 ... first byte A`（后半段为 B）或 `first byte B`（后半段为 A）
- 多次读：若干次 `read n`，各段首字节只会是 A 或 B（具体分段随调度与内核缓冲状态变化）

| (2) 会发生死锁吗？

通常不会形成严格意义死锁：

- 写者写完关闭写端并退出；父进程也关闭写端。
- 读者循环 `read`，当所有写端都关闭且管道数据耗尽，`read` 返回 0 结束。
- `wait` 等待子进程退出，不构成资源循环等待链。

补充：若某进程一直不关闭写端，读者可能在数据读完后阻塞于 `read`（更像“等待/阻塞”，不一定构成死锁定义里的环）。

（3）四个进程打印的 `shared` 值分别是多少？

`shared` 为普通进程内变量，`fork` 后各进程各有一份副本（写时复制），互不影响：

- `Writer0`: `shared = 1`
- `Writer1`: `shared = 2`
- `Reader`: 循环打印时 `shared` 仍为 `0`（其 `shared=100` 发生在循环之后且未打印）
- `Parent`: `shared` 一直为 `0`，最终打印 `Parent: shared = 0`

输出先后顺序不定（由调度决定）。

二、计算题（参考解）

时间片长度 2。优先级数值越小越高。到达时若更高优先级则抢占。

调度切分（按“发生抢占/时间片结束/进程结束”的边界）：

- `[0,1)`：P1 运行；`t=1` P2 到达且优先级更高，抢占
- `[1,3)`：P2 运行（剩余 2）
- `[3,5)`：P2 继续运行并完成（`t=3` P4 到达但优先级更低，不抢占）
- `[5,7)`：P1 运行
- `[7,9)`：P4 运行
- `[9,11)`：P1 运行
- `[11,13)`：P4 运行
- `[13,15)`：P1 运行
- `[15,17)`：P4 运行并完成
- `[17,18)`：P1 运行并完成
- `[18,20)`：P3 运行并完成

执行片段顺序：

P1 → P2 → P2 → P1 → P4 → P1 → P4 → P1 → P4 → P1 → P3

响应时间（到达→首次获得 CPU）：

- P1: 0
- P2: 0（到达即抢占）
- P4: 首次运行在 `t=7`，到达 `t=3`，响应时间 4
- P3: 首次运行在 `t=18`，到达 `t=2`，响应时间 16

平均响应时间： $(0 + 0 + 4 + 16) / 4 = 5$

公平性：

- 对低优先级不公平，可能饥饿（P3 长时间得不到 CPU）。
- 改进：老化（aging）。等待越久动态优先级逐步提高，保证长期等待进程最终能被调度。

三、编程题（参考解：信号量 P/V 伪代码）

目标约束：

- 调价修改 `price`，与任何其他互斥。
- 查询并发读 `seats/price`。
- 预订：读 `seats` 不阻塞查询；写 `seats--` 时与查询互斥；不超售；预订与调价互斥。

做法：两个“读写锁”（均用信号量实现）

- 全局 RW 锁 G：查询/预订 = 读者，调价 = 写者（保证调价独占）
- `seats` RW 锁 S：查询 = 读者；预订的“修改 `seats`”阶段 = 写者（保证 `seats` 写与查询互斥；读阶段不挡查询）

信号量/计数：

```
semaphore g_mutex = 1, g_wrt = 1;    // G 全局 RW 锁
int g_rcnt = 0;

semaphore s_mutex = 1, s_wrt = 1;    // S seats RW 锁
int s_rcnt = 0;
```

RW 锁操作：

```
// G: 全局读锁/写锁
G_read_lock() {
    P(g_mutex);
    g_rcnt++;
    if (g_rcnt == 1) P(g_wrt);
    V(g_mutex);
}
G_read_unlock() {
    P(g_mutex);
    g_rcnt--;
    if (g_rcnt == 0) V(g_wrt);
    V(g_mutex);
}
G_write_lock() { P(g_wrt); }
G_write_unlock() { V(g_wrt); }

// S: seats 读锁/写锁
S_read_lock() {
    P(s_mutex);
    s_rcnt++;
    if (s_rcnt == 1) P(s_wrt);
    V(s_mutex);
}
S_read_unlock() {
    P(s_mutex);
    s_rcnt--;
```

```

    if (s_rcnt == 0) V(s_wrt);
    V(s_mutex);
}
S_write_lock() { P(s_wrt); }
S_write_unlock() { V(s_wrt); }

```

查询进程：

```

Query() {
    G_read_lock();
    S_read_lock();
    int s = seats;
    int p = price;
    S_read_unlock();
    G_read_unlock();
    return (s, p);
}

```

调价进程（独占一切）：

```

AdjustPrice(newPrice) {
    G_write_lock();
    price = newPrice;
    G_write_unlock();
}

```

预订进程（读 seats 不挡查询；写 seats 独占；二次检查防超售）：

```

Book() {
    G_read_lock();           // 防止调价并发（预订与调价互斥）

    int p = price;           // 读 price: 允许与查询并发

    S_read_lock();           // seats 读阶段：不阻塞查询
    int s = seats;
    S_read_unlock();
    if (s <= 0) { G_read_unlock(); return FAIL; }

    S_write_lock();          // seats 写阶段：与查询互斥；预订间串行
    if (seats > 0) {          // 二次检查：并发预订下防超售
        seats--;
        S_write_unlock();
        G_read_unlock();
        return SUCCESS(p);
    } else {
        S_write_unlock();
        G_read_unlock();
        return FAIL;
    }
}

```

